Food Ordering System Implementation

Using Builder Design Pattern

Solving object creation complexity in customizable systems through elegant design patterns

The Challenge of Modern Customization

Today's users demand **high levels of customization** in their food orders — from meal types and sizes to specific toppings and extras.

Manually creating these complex objects leads to what developers call **"Constructor Hell"** — when constructors accumulate 10+ parameters and become impossible to manage.

What is the Builder Design Pattern?

Step-by-Step Construction

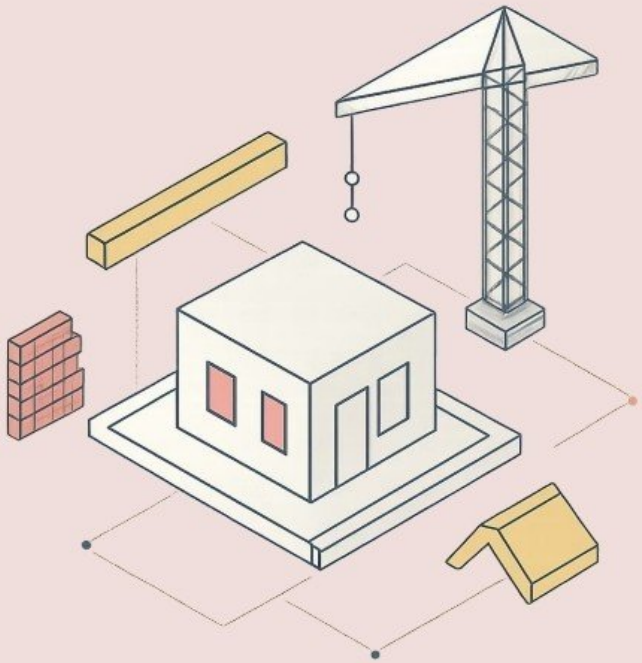
Build complex objects incrementally, setting only the features you need for each specific order

Separation of Concerns

Separates the construction logic from the object representation for cleaner, more maintainable code

Flexible & Readable

Creates fluent, menu-like code that's easy to understand and extend with new options



Why Builder Pattern Matters

1

Eliminates Long Parameter Lists

No more confusing constructors with 10+ parameters. Each option is set explicitly with clear method names.

2

Improves Code Readability

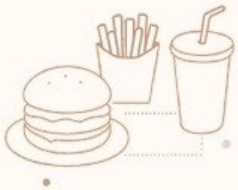
It becomes immediately clear which value belongs to which ingredient, making the code self-documenting.

3

Enhances Maintainability

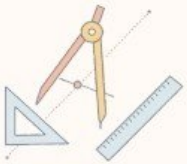
Adding new options like "Extra Spicy" doesn't break existing code—just extend the builder methods.

Core Components of the Project



Meal.java

The product being built with private fields for type, size, drink, and extras



MealBuilder.java

The engine handling construction steps with fluent method chaining

MealDirector.java

Defines specific recipes and standardized meal combinations

Main.java

The client execution point where the system comes to life

Meal.java: The Product

Structure

The Meal class represents the final product being constructed. It contains private fields for all customizable attributes:

```
private String type;
private String size;
private String drink;
private String extras;
```

These fields are set through the builder and can be displayed using a `displayMeal()` method.

MealBuilder.java: The Engine

The builder handles the step-by-step construction process using method chaining:

Fluent Interface

Each setter method returns `this`, allowing method chaining:

```
public MealBuilder setType(String type) {
    meal.setType(type);
    return this;
}
```

Build Method

Returns the final constructed Meal object:

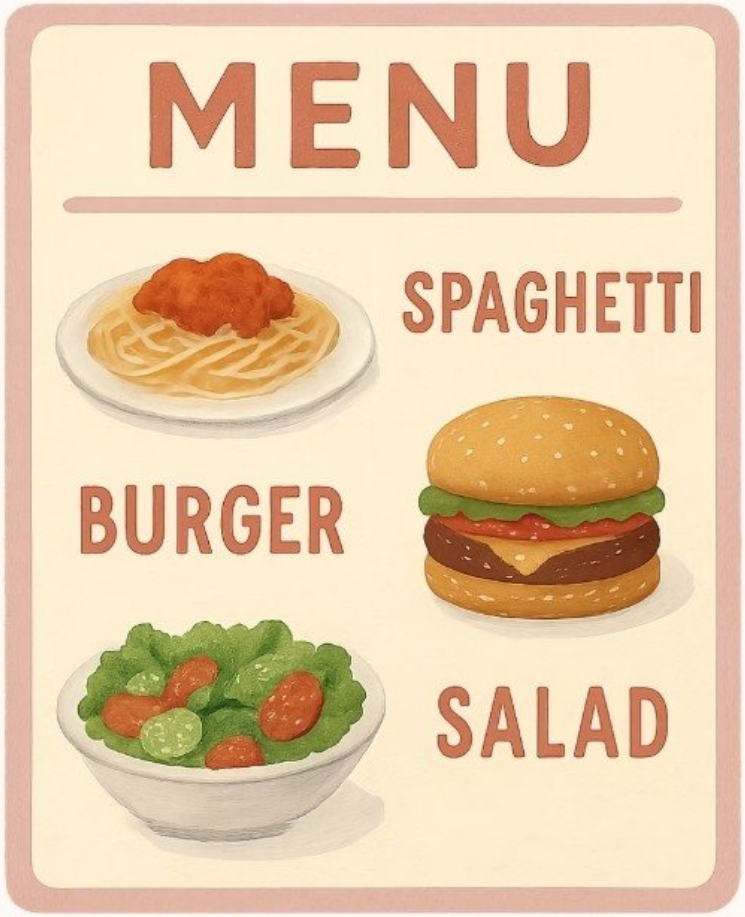
```
public Meal build() {
 return meal;
}
```

MealDirector.java: The Manager

Defining Recipes

The director class defines specific meal configurations. It encapsulates common meal patterns:

```
public Meal makeBurgerMeal() {
    return new MealBuilder()
        .setType("Burger")
        .setSize("Large")
        .setExtras("Cheese")
        .setDrink("Cola")
        .build();
}
```



Advantages & Benefits

Precise Control

Provides fine-grained control over the construction process, allowing customization at each step

Code Reusability

Same construction process creates different meal types, reducing code duplication and maintenance

Enhanced Readability

Code reads like a menu — `.addSauce().addDrink()` makes intent crystal clear

Considerations & Trade-offs

Complexity

Requires creating multiple new classes (Builder, Concrete Builder, Director), which can feel like overkill for simple objects.

Overhead

For very basic objects with few parameters, the pattern might add unnecessary lines of code and complexity.

Best suited for objects requiring many optional parameters or complex construction logic

Key Takeaways

01

When to Use Builder

Apply when objects have many optional parameters or complex construction logic that varies by use case

02

Separation is Key

Always separate construction from representation to maintain clean, modular code architecture

03

Fluent Interface

Method chaining creates readable, menu-like code that developers love to work with

04

Plan for Growth

Builder pattern makes it easy to add new options without breaking existing client code

Project Team

Team Member	Responsibilities
Asmaa Elaywi	Team Leader, GitHub Manager, README & UML diagram
Bylasan Hamdan	Presentation & Media Specialist
Fatima alzhraa Aljnaidah	Lead Developer & Technical Presenter
Ruba Alkum	System Analyst & Concept Presenter